ASSIGNMENT

TECHNOLOGY PARK MALAYSIA

CT127-3-2-PFDA

PROGRAMMING FOR DATA ANALYSIS

APD2F2302SE

HAND OUT DATE: 08 MAY 2023
HAND IN DATE: 26 MAY 2023
WEIGHTAGE: 50%
STUDENT NAME: AISHATH SHAFA SAUDULLA
STUDENT TP: TP066434

INSTRUCTIONS TO CANDIDATES:

1. **Submit your assignment at the administrative counter.**
2. **Students are advised to underpin their answers with the use of references (cited using the American Psychological Association (APA) Referencing).**
3. **Late submissions will be awarded zero (0) unless Extenuating Circumstances (EC) are upheld.**
4. **Cases of plagiarism will be penalized.**
5. **The assignment should be bound in an appropriate style (comb bound or stapled).**
6. **Where the assignment should be submitted in both hardcopy and softcopy, the softcopy of the written assignment and source code (where appropriate) should be on a CD in an envelope / CD cover and attached to the hardcopy.**
7. **You must obtain 50% overall to pass this module.**

Table of Contents

1. Introduction and assumptions 3
2. Data import / Cleaning / pre-processing / transformation 4
2.1 Data Import 4
2.2 Data Cleaning 5
2.3 Data Pre-Processing 5
2.4 Data Transformation 5
2.5 Data Exploration 6
3. Questions 7
3.1 Histogram: How many employees exist in the age groups. 7
3.2 Scatterplot: Relationship between the reasons for termination and the length of service of employees 9
3.3 Boxplot Analysis: Gender vs Top 12 Departments 11
3.4 Violin Plot: Age distribution among employees in the top 3 cities with the highest frequencies 13
3.5 Boxplot Analysis: Age by Department (Bottom 12) 15
3.6 Dot Chart: Comparing the job to how many employees exist in the job titles 17
3.7 Bar Plot: Analysis to see all business unit's ratio of employees 19
3.8 Strip Plot: of Hired Dates over the years (STHDA, n.d.-b) 21
3.9 Bar Graph: Count of Employee ID by Hiring Year 23
3.10 Line Plot: Status by Status Year for Employees 25
3.11 Stacked Bar Graph: Length of Service by Gender fill 27
3.12 Boxplot: Length of Service by Department 29
3.13 Stacked Histogram: Employee Age Distribution 31
3.14 Bar Graph: Employee Birth Month Distribution 33
3.15 Bar Graph: Employee count in top 10 stores 35
4. Extra/Additional Features 37
4.1 Pie Chart: Top 5 departments and their employee count 37
4.2 Treemap: Count of Employees in City's 39
4.3 Lollipop Chart: Number of Employees by Department 41
4.4 Moving Scatterplot: Employee Length of Service vs Age 43
5. Conclusion 45
6. References 45

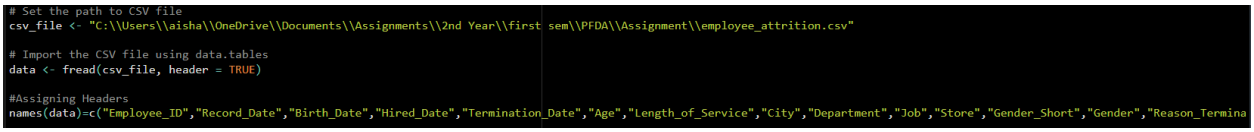
1. Introduction and assumptions

I explore employee attrition, or workers leaving a company, in this examination. I can find patterns and trends connected to attrition by looking at a dataset with employee information, which enables organizations to create plans for staff retention and enhance overall performance. I rely on the dataset to be accurate and representative of the full employee population. Organizations may pinpoint areas for development, put specific measures in place to deal with attrition issues, and boost employee retention. By identifying high-risk regions and allocating resources accordingly, workforce planning may be optimized. Additionally, the study can help in the formulation of programs for employee retention that are specifically catered to their requirements and preferences, improving the work environment, and enhancing organizational performance.

2. Data import / Cleaning / pre-processing / transformation

DATA IMPORT- how you imported your dataset. DATA CLEANING-(Its for removing the junk values, remove NA values) . DATA PREPROCESSING- (if you have changed the column headings, datatypes, changes values from Y/N to 0/1 etc) DATA EXPLORATION-(the very basic information about your dataset like what are the titles, no:of rows/cols, head/tail functions....(Esp: first topic of R class)

2.1 Data Import



I set the path for the csv file to be imported as you can see in the above screenshot. Then I use the set path to import the data. I use fread as it is a large data set it is recommended. Then I assign my own chosen headers for the data set to make it easier when coding later. The assigned header names consist of:

1) Employee_ID
2) Record_Date
3) Birth_Date
4) Hired_Date
5) Termination_Date
6) Age
7) Length_of_Service
8) City
9) Department
10) Job
11) Store
12) Gender_Short
13) Gender
14) Reason_Termination
15) Type_Termination
16) Status_Year
17) Status
18) Business_Unit"

2.2 Data Cleaning

```
# Remove duplicate records based on employee ID
data <- unique(data, by = "Employee_ID")
```

For data cleaning I make sure that there is no data redundancy by ensuring that all Employee IDs are unique. I have also used tidy verse to cleanse the data automatically. Tidy verse is a powerful tool consisting of a collection of packages for data science (Bates, 2020). I added the tidy verse library at the beginning of the code with all the other packages.

2.3 Data Pre-Processing

```
# Perform encoding for job titles
encoded_titles <- data.table::dcast(data, Employee_ID + Age ~ Job, value.var = "Job", fun.aggregate = length, fill = 0)

# Convert data$Age to numeric if it is not already numeric
if (!is.numeric(data$Age)) {
  data$Age <- as.numeric(data$Age)
}

# Create age groups
data$age_group <- cut(data$Age, breaks = c(0, 20, 30, 40, 50, Inf), labels = c("Under 20", "20-29", "30-39", "40-49", "50+"))
```

As I had mentioned before in 2.1, I had changed the headers to simpler names. According to Bates, 2020 Tidy verse does work in pre-processing as well. I have attempted to create a mini table with just the employee ID and age as rows and job titles as columns.

I also created age groups as the list of ages are quite long. To create the age groups, I had to change the age data to numeric beforehand for the parts that were not already numeric.

2.4 Data Transformation

```
# Create a new variable for month of birth
# Convert data$Birth_Date to date format
data$Birth_Date <- as.Date(data$Birth_Date)

# Extract the birth month using lubridate::month() to create the variable
data$birth_month <- lubridate::month(data$Birth_Date, label = TRUE)
```

For data transformation I have created a new variable month of birth. To do so I had to converse the birth date to date format first and then extract the birth month using lubridate to create this variable.

2.5 Data Exploration

```
# Viewing methods

#view only 5
head(data, 5)

#view only title\heading
names(data)

#how data stored
class(data)

#summary min/max/average
summary(data$Age)
summary(data)

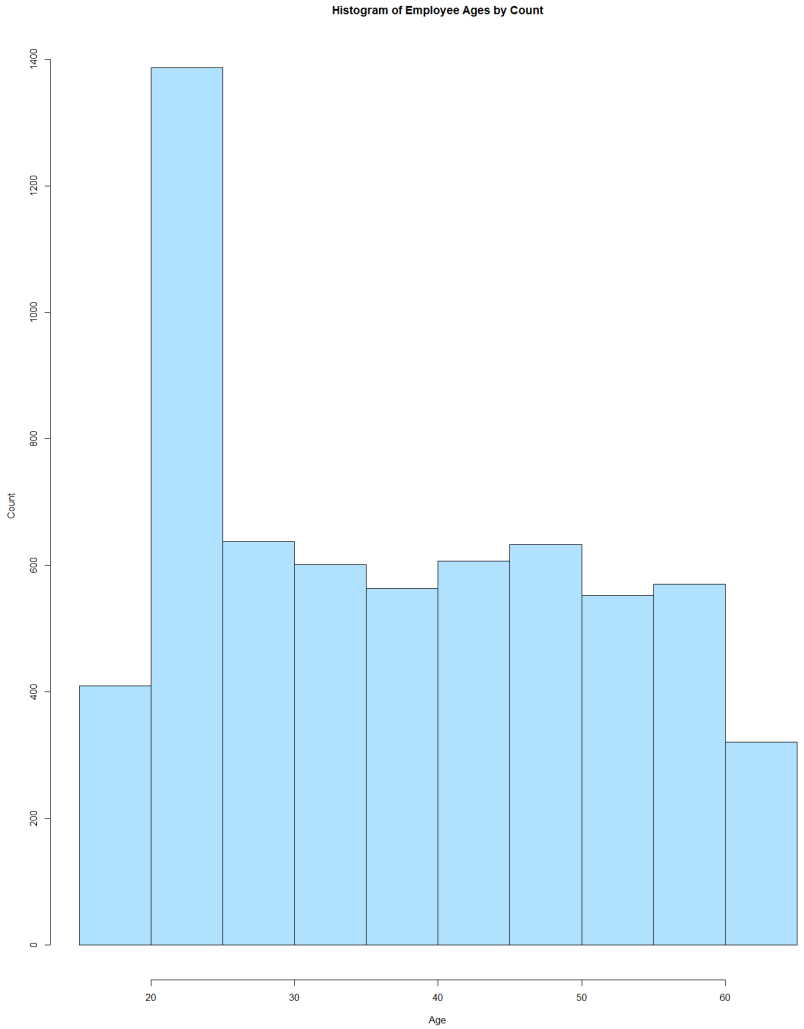
#glimpse of data
glimpse(data)
```

I have added some viewing methods to make it easier while exploring the data including viewing only the top 5 data, viewing only the titles, seeing how data class, summary's of data and a glimpse into the data as well.

3. Questions

3.1 Histogram: How many employees exist in the age groups.

```
# Plot a histogram for age
hist(data$Age, col = "#B0E2FF", border = "black",
     xlab = "Age", ylab = "Count",
     main = "Histogram of Employee Ages by Count")
```



Code Explanation:

The hist() function is used to create this histogram (Schork, n.d.) .data$Age specifies the "Age" column from the dataset, which represents the ages of the employees. col sets the color of the histogram bars to "#B0E2FF" (a light blue shade) (R-Charts, n.d.). border sets the color of the borders of the histogram bars to black. xlab and ylab specify the labels for the x-axis and y-axis, respectively. main sets the title of the histogram to "Histogram of Employee Ages by Count".

Analysis Aim:

The aim of this analysis is to visualize the distribution of the employee ages in the dataset Employee Attrition and to answer the question of how many employees do exist in the different age groups throughout the employees.

Analysis Description:

The histogram displays the distribution of all the employee ages. The x-axis represents different age values, and the y-axis represents the count of employees falling within each age group. The height of each bar represents all the number of employees in that specific age group.

Justification/Conclusion:

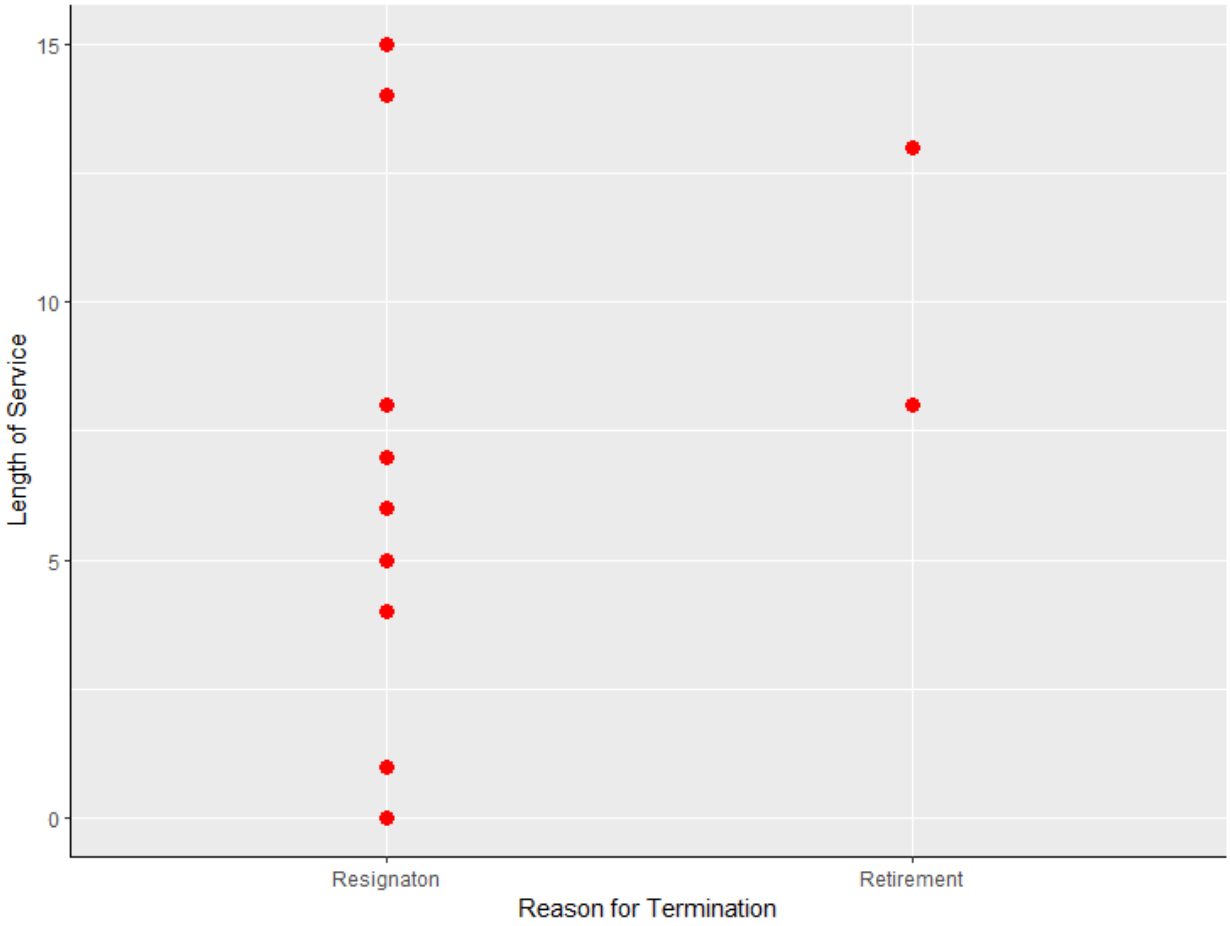
By examining the histogram, we can analyze the distribution of employee ages. We can identify the count of all the employees in different age groups, which helps us understand the age composition of the company’s employees. This information can be valuable for various HR-related analysis, which may require to identify succession planning, workforce diversity, or age-related policies or various other decision-making processes.

3.2 Scatterplot: Relationship between the reasons for termination and the length of service of employees

```
filtered_data <- data %>%
  filter(Reason_Termination != "Not Applicable")

ggplot(filtered_data, aes(x = Reason_Termination, y = Length_of_Service)) +
  geom_point(color = "red", size = 3) +
  xlab("Reason for Termination") +
  ylab("Length of Service") +
  theme(axis.line = element_line(colour = "black", size = 0.24))
```

(R-Coder, 2020)



Code Explanation:

The reason for termination has a lot of not applicable in it so to filter it out I created "filtered_data" to exclude rows where the "Reason_Termination" is "Not Applicable". This step removes any irrelevant or non-applicable data from this analysis. Then to create a scatter plot with the filtered data the x-axis represents the "Reason_Termination" variable, while the y-axis represents the "Length_of_Service" variable. Each data point is represented as a red point of a size 3. The x-axis label is set to "Reason for Termination" using xlab(), and the y-axis label is set to "Length of Service" using ylab(). Additionally, the theme() function is used to modify the appearance of the plot by setting the color “red” and size “3” of the axis lines.

Analysis Aim:

The aim of this analysis is to understand the relationship between the reasons for termination and the length of service for employees in the company.

Analysis Description:

The r code filters out all rows with "Not Applicable" as the reason for termination and creates a scatter plot with red dots to visualize the relationship between the reason for termination and the length of service in the dataset Employee Attrition.

Justification:

Filtering out the "Not Applicable" records allows us to focus on relevant termination reasons for analysis. By creating a scatter plot, we can visually examine the distribution and potential patterns or trends in the data which is typically use in HR. This data analysis is useful to see what the reason for termination could possibly be and so on.

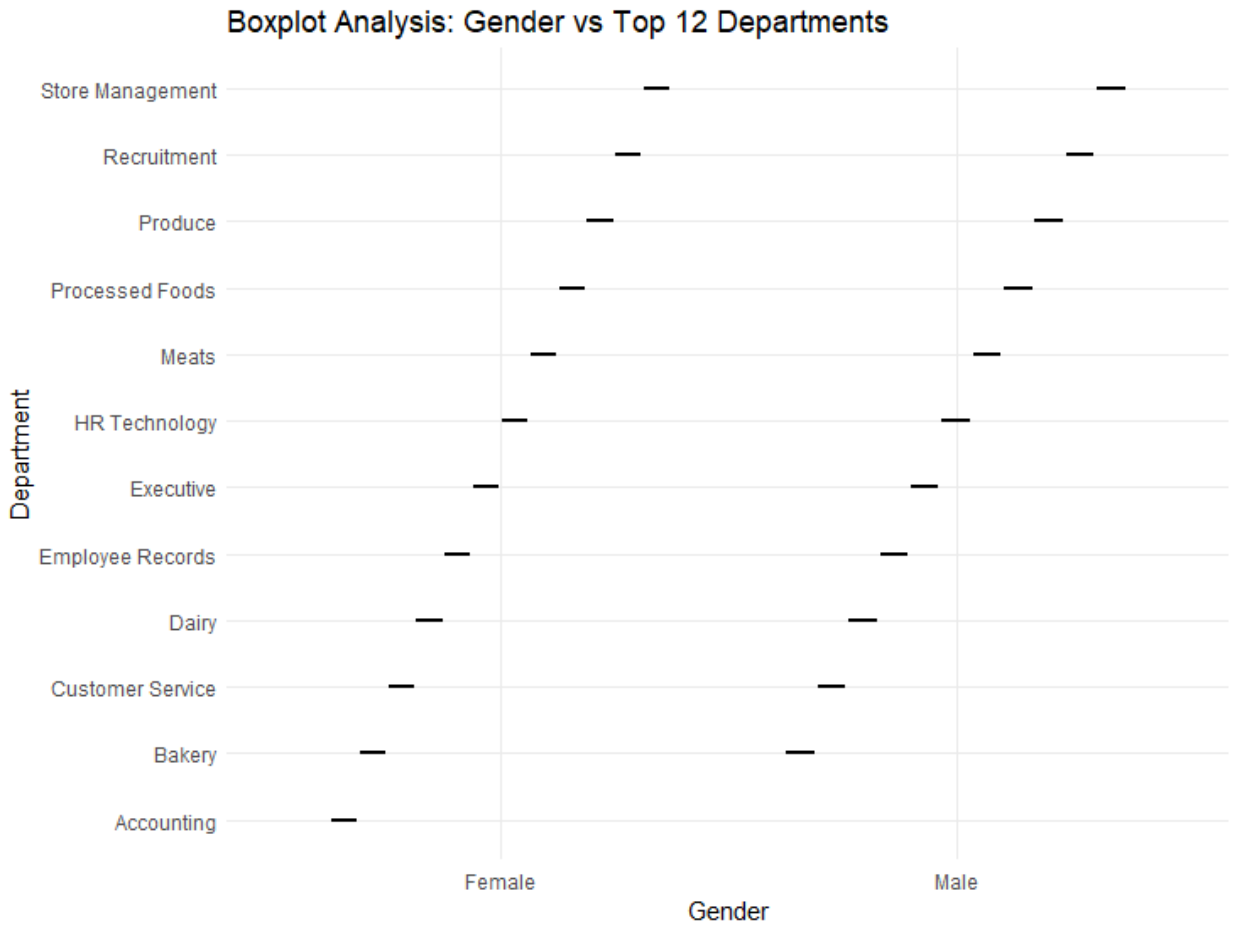
Conclusion:

Based on the scatter plot, we can observe the distribution of length of service for different reasons for termination. The red points represent individual employees, and their positions on the plot indicate the specific reason for termination and their corresponding length of service. Further analysis and interpretation of the plot may reveal insights into whether certain termination reasons are more common among employees with specific lengths of service and can draw a calculative summary using statistics to conclude whether it is all relevant to one another and draw out the patterns.

3.3 Boxplot Analysis: Gender vs Top 12 Departments

```
# Filter the data for the top 12 departments
top_departments <- head(sort(table(data$Department), decreasing = TRUE), 12)
filtered_data <- data[data$Department %in% names(top_departments), ]

# Create the boxplot
ggplot(filtered_data, aes(x = Gender, y = Department)) +
  geom_boxplot(fill = "white", color = "black") +
  xlab("Gender") +
  ylab("Department") +
  ggtitle("Boxplot Analysis: Gender vs Top 12 Departments") +
  theme_minimal()
```



Code Explanation:

I first filtered the departments to include only the top 12 departments based on their frequency or count in the Employee Attrition dataset. It calculates the frequency of each department using table(data$Department) and then sorts them in the descending order with sort(), takes the top 12 with head(), and stores the result in top_departments. Then, filtered_data is created by subsetting the original data to include only the rows where the department is one of the top 12 in the list.

Then I create the boxplot visualization which specifies the filtered_data as the data source for the plot. The aes() function defines the mapping, with Gender as the x-axis variable and Department as the y-axis variable. The geom_boxplot() function is used to create the boxplot, and options like fill and color set out the look of the plot. The xlab(), ylab(), and ggtitle() functions set the labels for the x-axis, y-axis, and plot title, respectively. Finally, theme_minimal() adjusts the plot's theme.

Analysis Aim:

To visually compare the distribution of genders across the top 12 departments in the employee attrition dataset.

Justification:

By examining the gender distribution across departments, we can gain insight into the potential gender imbalances or variations in attrition rates across different areas of the organization.

Conclusion:

The boxplot visualization provides a view of the gender distribution in the top 12 departments where each department is represented by a line boxplot, with the y-axis showing the department names and the x-axis representing the gender. We can check for any significant differences, discrimination, and similarities in gender representation across the top 12 departments.

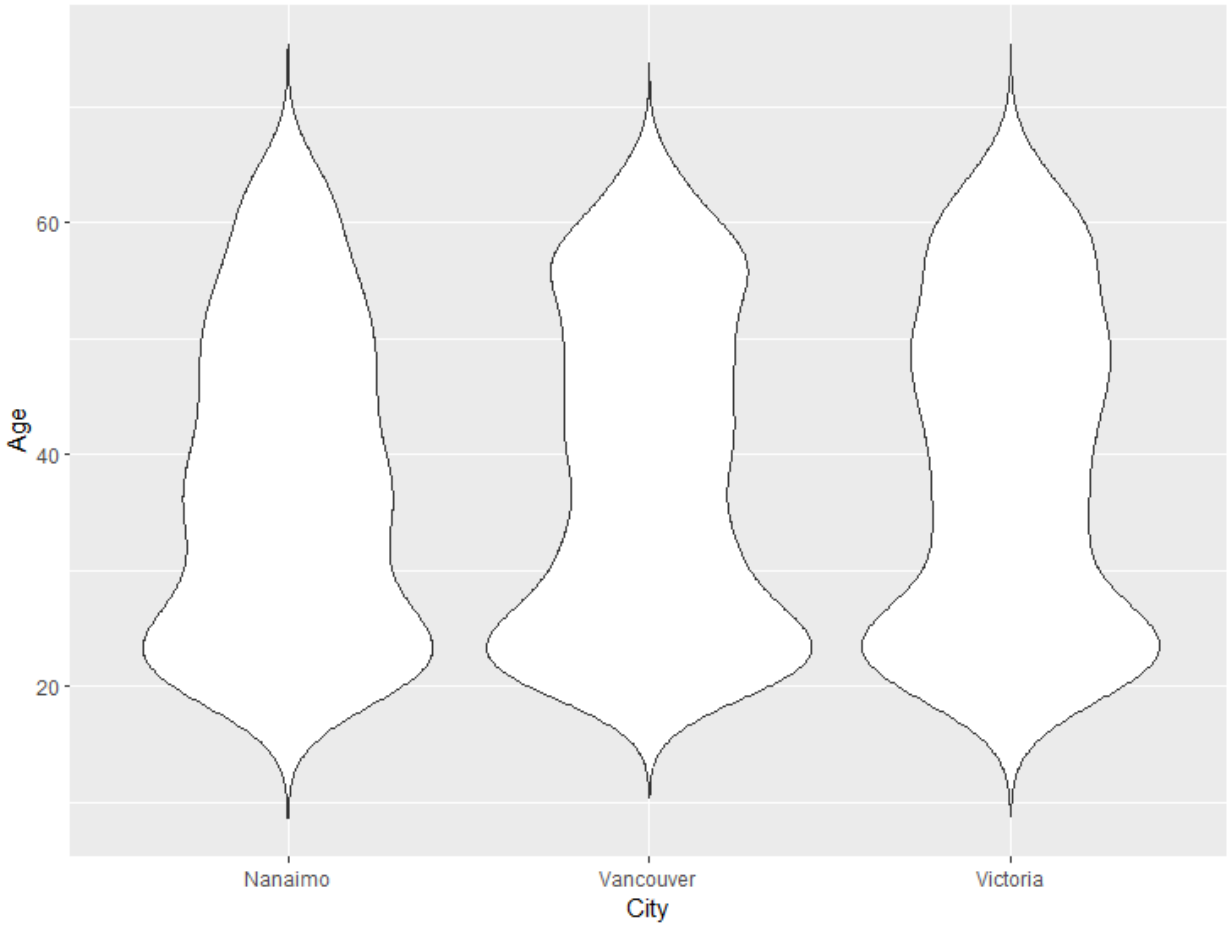
3.4 Violin Plot: Age distribution among employees in the top 3 cities with the highest frequencies

```
top_city <- head(sort(table(data$City), decreasing = TRUE), 3)
filtered_city <- data[data$City %in% names(top_city), ]

# Basic violin plot
p <- ggplot(filtered_city, aes(x = City, y = Age)) +
  geom_violin()
p

# Rotate the violin plot
p + coord_flip()

# Set trim argument to FALSE
ggplot(filtered_city, aes(x = City, y = Age)) +
  geom_violin(trim = FALSE)
```



Code Explanation:

top_city <- head(sort(table(data$City), decreasing = TRUE), 3)` counts the frequency of each city in the "City" column of the dataset and sorts the cities in descending order based on their frequency and selects the top 3 cities with the highest frequencies and stores them in the `top_city` variable. Creating a new dataset called `filtered_city` containing only the filtered rows. Then using ggplot2 I plot using the `geom_violin()` function. (STHDA, n.d.).

Analysis Aim:

To explore the age distribution among employees in the top 3 cities with the highest frequency or count.

Justification:

By examining the age distribution within the top 3 cities, we can gain insight into potential age-related patterns or differences in employee attrition.

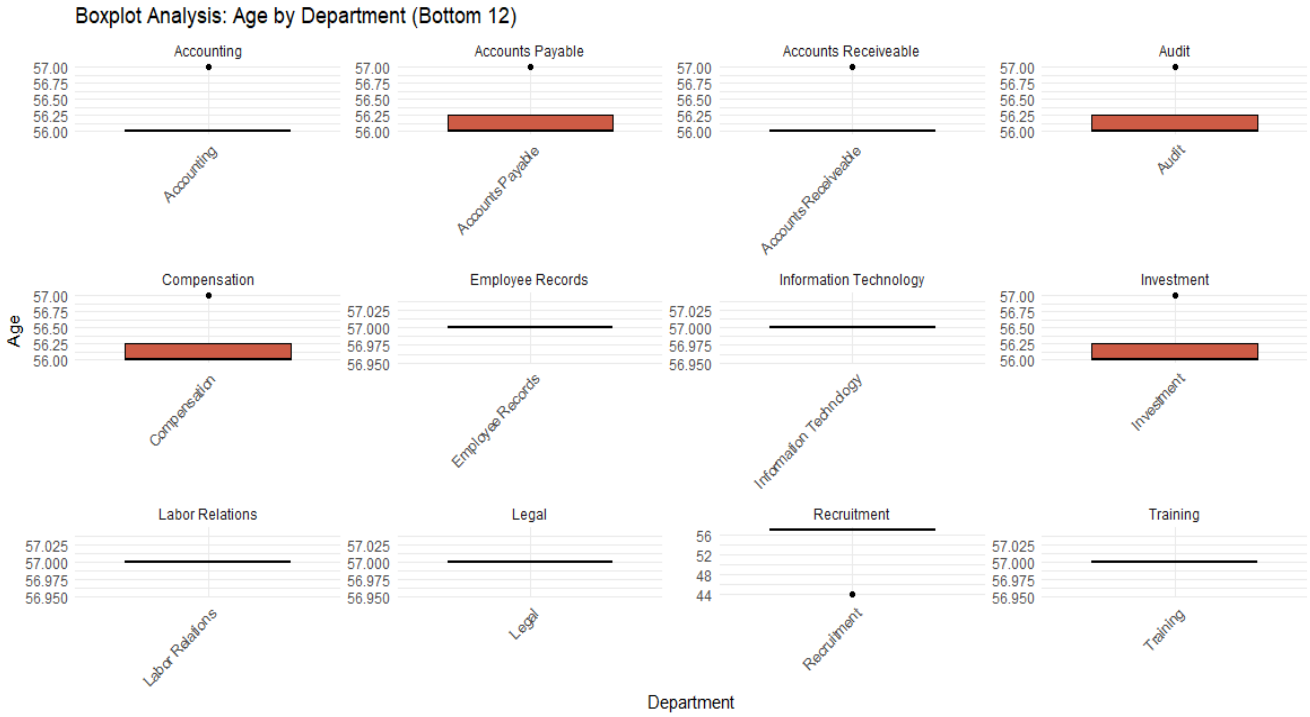
Conclusion:

From the violin plots, we can observe the following whether the age distribution varies across the top 3 cities, the width of the violins indicates the density of employees ages within each city. Overall, this analysis provides a visual representation of the age distribution among employees in the top 3 cities with the highest count.

3.5 Boxplot Analysis: Age by Department (Bottom 12)

```
# Filter the data for the bottom 12 departments
bottom_departments <- tail(sort(table(data$Department), decreasing = TRUE), 12)
filtered_data <- data[data$Department %in% names(bottom_departments), ]

# Create the boxplot with separate plots for each department
ggplot(filtered_data, aes(x = Department, y = Age)) +
  geom_boxplot(fill = "#CD5B45", color = "black") +
  xlab("Department") +
  ylab("Age") +
  ggtitle("Boxplot Analysis: Age by Department (Bottom 12)") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  facet_wrap(~ Department, scales = "free")
```



Code Explanation:

bottom_departments <- tail(sort(table(data$Department), decreasing = TRUE), 12) calculates the frequencies of each department in the dataset and sorts them in descending order and then selects the bottom 12 departments with the lowest counts. Then, the filtered_data <- data[data$Department %in% names(bottom_departments),] filters the original data to include only the rows where the department is one of the bottom 12 departments. (*Chapter 11: Boxplots and Bar Graphs,* n.d.).

Analysis Aim:

To examine the distribution of employee ages across the bottom 12 departments in the dataset as the bottom departments is believed to hold the more crucial positions in the company.

Justification:

Analyzing the age distribution by department can provide insights into potential differences or similarities in age profiles across different departments. This information can be useful for understanding workforce demographics and potentially identifying areas where age-related considerations may be relevant especially in the job titles which hold more power compared to the other jobs.

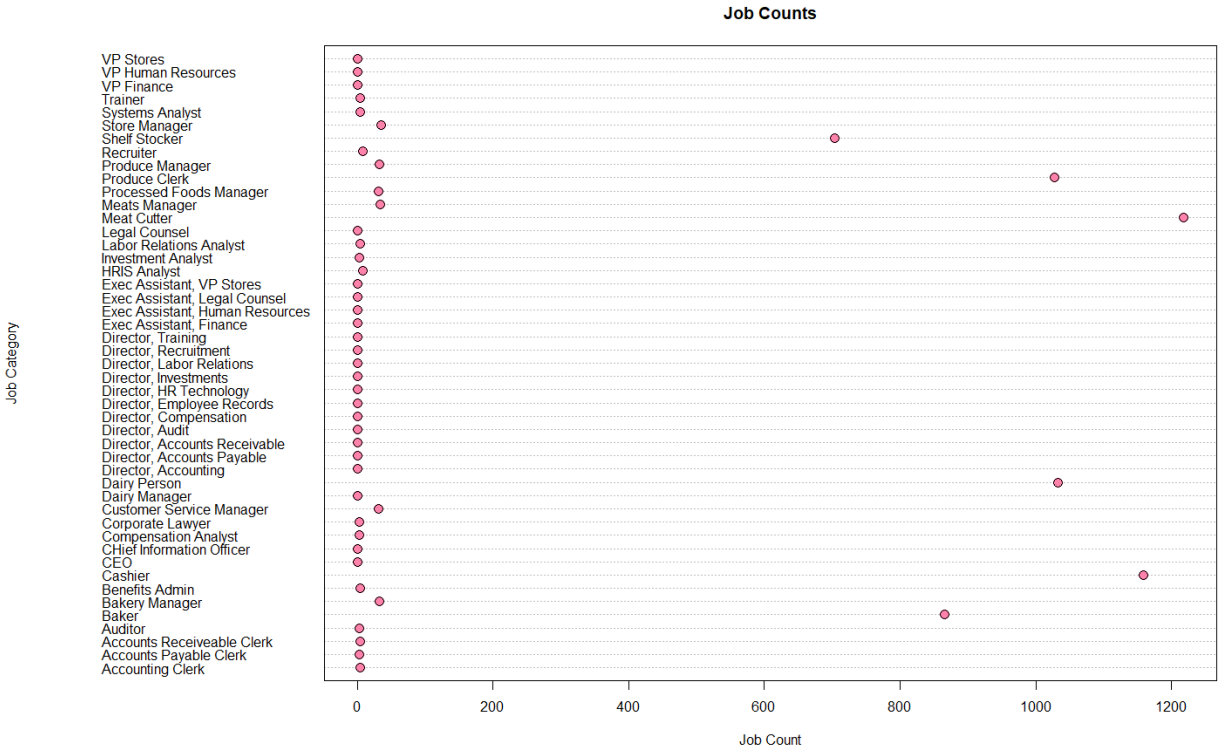
Conclusion:

This box plot displays a show of the distribution of various employee ages within each department, with the boxes representing the interquartile range (IQR) and the median, and the whiskers indicating the range of the data. The departments are displayed separately, allowing for visual comparison of age distributions between departments. The title of the plot indicates that the analysis is focused on the bottom 12 departments, which are the departments with the lowest frequencies in the dataset. Overall, this analysis provides a visual representation of the age distribution for each department within the bottom 12 departments. It can help identify any notable differences or similarities in the age profiles of employees across these departments in the employee attrition.

3.6 Dot Chart: Comparing the job to how many employees exist in the job titles

```
# Count the number of job rows for each job category
job_counts <- table(data$Job)

# Create the dot chart with job counts
dotchart(job_counts, labels = names(job_counts), pch = 21, bg = "#FF82AB", pt.cex = 1.5,
         xlab = "Job Count", ylab = "Job Category", main = "Job Counts")
```



Code Explanation:

The code counts the number of rows for each job category in the "Job" column of the "data" dataset using the `table()` function and assigns the result to the variable `job_counts`. The `dotchart()` function is then used to create a dot chart, where each dot represents a job category, and the height of the dot corresponds to the count of employees in the jobs in that category. The `labels = names(job_counts)` parameter specifies the labels for each dot, which are the job categories. The `pch = 21` and `bg = "green"` parameters set the shape and fill color of the dots. The `pt.cex = 1.5` parameter controls the size of the dots. The `xlab`, `ylab`, and `main` parameters specify the labels for the x-axis, y-axis, and the title of the plot, respectively. Finally, the `text()` function is used to add the job counts next to each dot on the plot. (R-Coder, 2020).

Analysis Aim:

To visualize the distribution of job categories and their employee counts in the "Employee Attrition" dataset.

Justification:

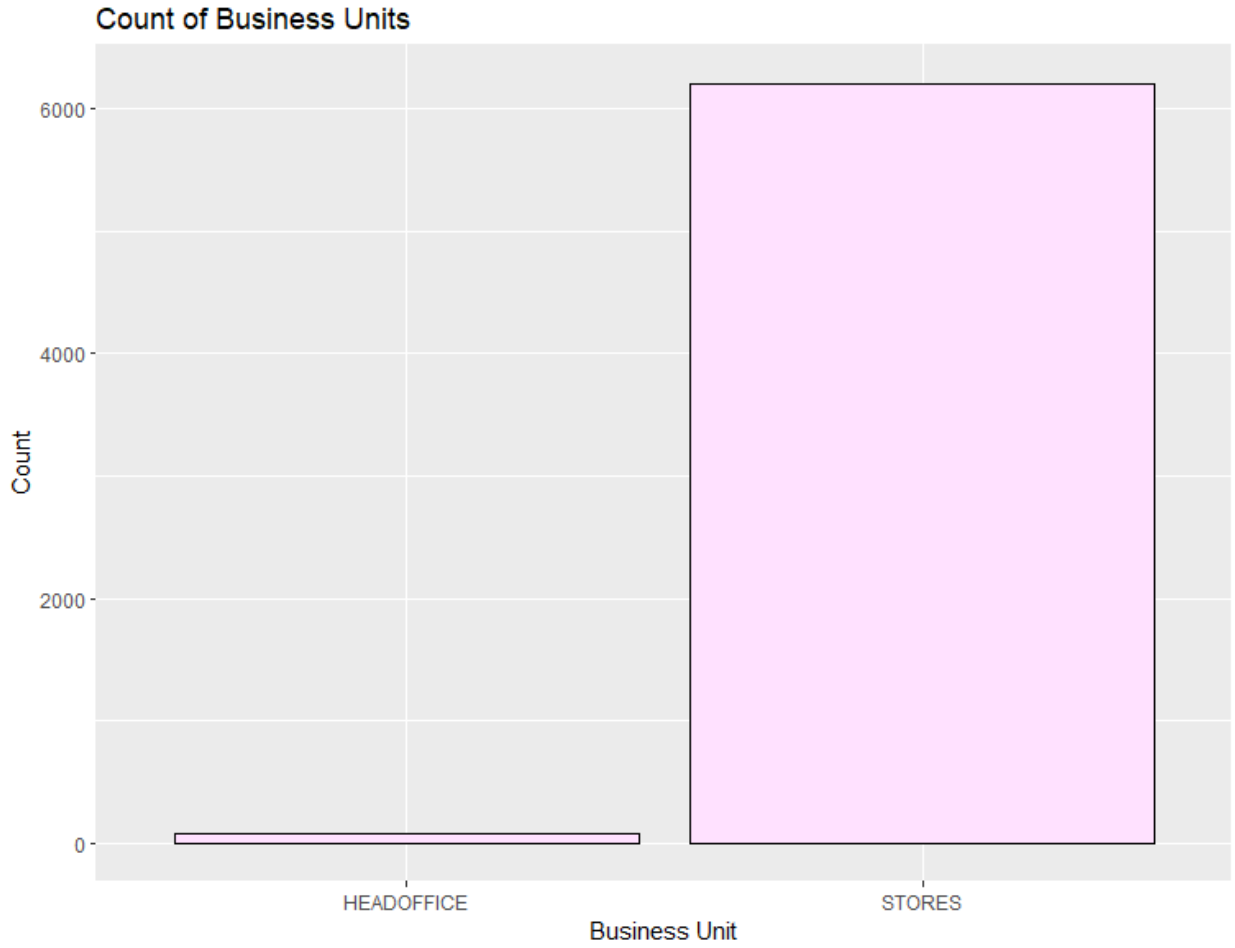
Using a dot chart is an effective way to display the count of job categories, as it allows for easy comparison and identification of the most and least common job categories, the addition of labels with the job counts provides easy information for each category.

Conclusion:

The height of each dot represents the count of jobs in that category. The analysis provides a clear visual representation of the distribution and allows for quick identification of the job categories with the highest and lowest counts. It aids in understanding the composition of job categories and their relative frequencies of each category. The analysis can be useful for identifying areas of the workforce with high or low representation and for making informed decisions related to recruitment, retention, and organizational planning and may be mainly used in HR departments in companies.

3.7 Bar Plot: Analysis to see all business unit's ratio of employees

```
ggplot(data, aes(x = Business_Unit)) +
  geom_bar(fill = "thistle1", color = "black") +
  labs(x = "Business Unit", y = "Count", title = "Count of Business Units")
```



Code Explanation:

To create a bar plot of the "Business_Unit" variable from the Employee Attrition dataset, I set the x-axis as "Business_Unit" and uses the geom_bar() function to represent the count of each business unit as bars. The fill color is set to "thistle1" (R-Charts, n.d.) and the bar outlines are colored black. The labs() function is used to set the x-axis label as "Business Unit", the y-axis label as "Count", and the plot title as "Count of Business Units". (Schork, n.d.-a)

Analysis Aim:

To visualize the distribution of employees across different business units within the dataset, Employee Attrition.

Justification:

By creating a bar plot, we can easily observe the count of employees in each business unit. The use of different colors for fill and outlines enhances the visual appeal and readability of the bar plot.

Conclusion:

The analysis of the employee attrition dataset revealed the distribution of employees across different business units. The generated bar plot successfully visualized the count of employees in each business unit, providing a clear understanding of the workforce distribution. The analysis aimed to gain insights into the composition of the workforce, which can be useful for further investigations and decision-making regarding resource allocation, departmental performance, and potential areas for improvement commonly used in Human Resources. As you can see, the head office has a very low count of employees in comparison to the stores as expected.

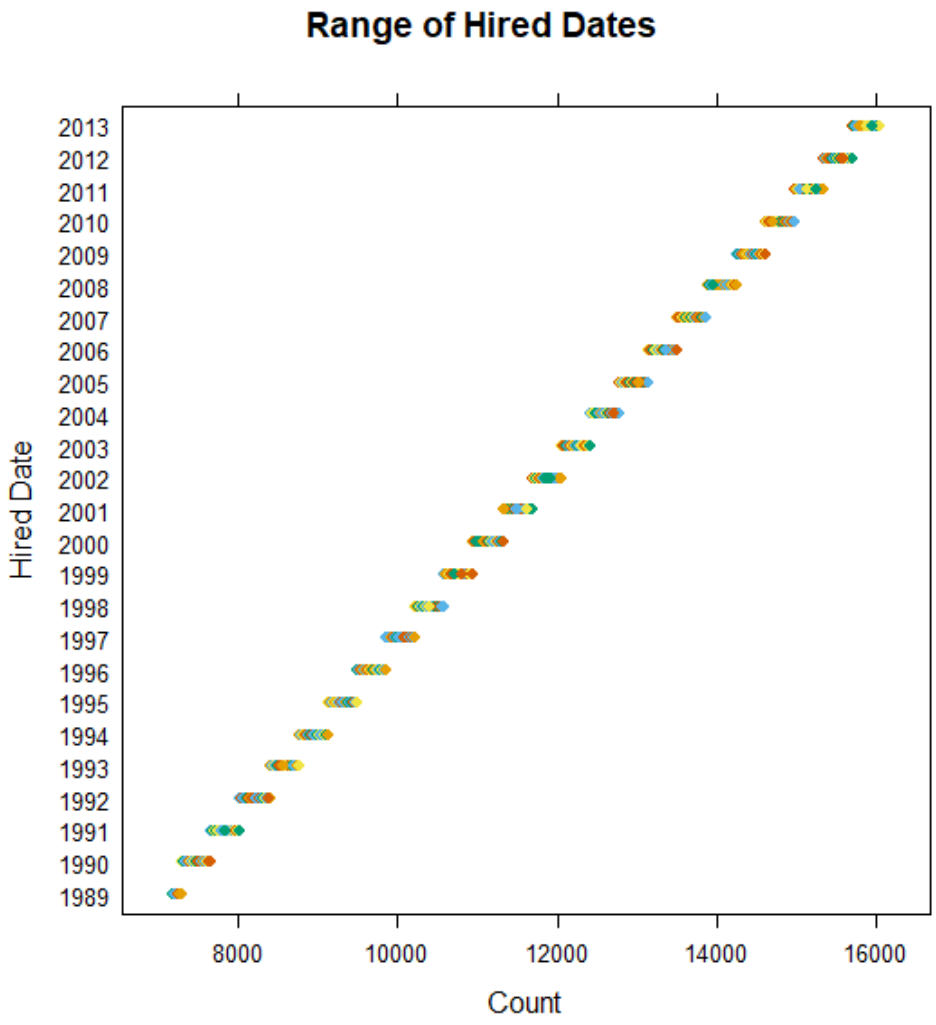
3.8 Strip Plot: of Hired Dates over the years (STHDA, n.d.-b)

```
# Convert Hired_Date to Date format
data$Hired_Date <- as.Date(data$Hired_Date, format = "%m/%d/%Y")

# Extract year from Hired_Date
data$Hired_Year <- format(data$Hired_Date, format = "%Y")

# Set individual colors for each year
colors <- c("#E69F00", "#56B4E9", "#F0E442", "#009E73", "#D55E00")

# Create strip plot with separate colors for each year
stripplot(Hired_Year ~ as.numeric(Hired_Date), data = data,
          pch = 16, col = colors, aspect = 1,
          main = "Range of Hired Dates", xlab = "Count", ylab = "Hired Date")
```



Code Explanation:

First I convert the "Hired_Date" column in the "data" dataset to a Date format using the `as.Date()` function.Then, extract the year from the "Hired_Date" column and create a new column called "Hired_Year" using the `format()` function.A vector called "colors" is defined, which contains individual colors to be used for each year in the plot. The `stripplot()` function is used to create a strip plot, where the "Hired_Year" is plotted against the numeric representation of "Hired_Date". The "data" parameter specifies the dataset as mentioned before. The "pch" parameter sets the shape of the points in the plot. The "col" parameter sets the colors for each year. The "aspect" parameter adjusts the aspect ratio of the plot. The "main" and "xlab" parameters set the plot title and x-axis label, respectively. The y-axis label is set to "Hired Date". (STHDA, n.d.-b).

Analysis Aim:

To visualize the range of hired dates for employees, categorizing them by the year of their hiring.

Justification:

By visualizing the distribution of hired dates across different years, we can identify any patterns or trends in the hiring patterns within the organization which inturn helps in understanding the distribution of hiring over time and identifying any shifts or variations in the recruitment process.

Conclusion:

From the strip plot, we can observe the distribution of hired dates across different years. Each point represents an employee's hiring date, and the colors indicate the respective years. This visualization allows us to identify any concentration or gaps in hiring for specific years. It can also provide insights into the growth or fluctuations in recruitment activities over time. The analysis of the range of hired dates provides valuable information about the organization's hiring patterns. By visualizing the distribution of hiring across different years, we can gain insights into the trends and understand the dynamics of the recruitment process which Human Resource and the Hiring Team will find very useful in the long run. This information can be useful for identifying recruitment strategies, and evaluating the effectiveness of hiring efforts.

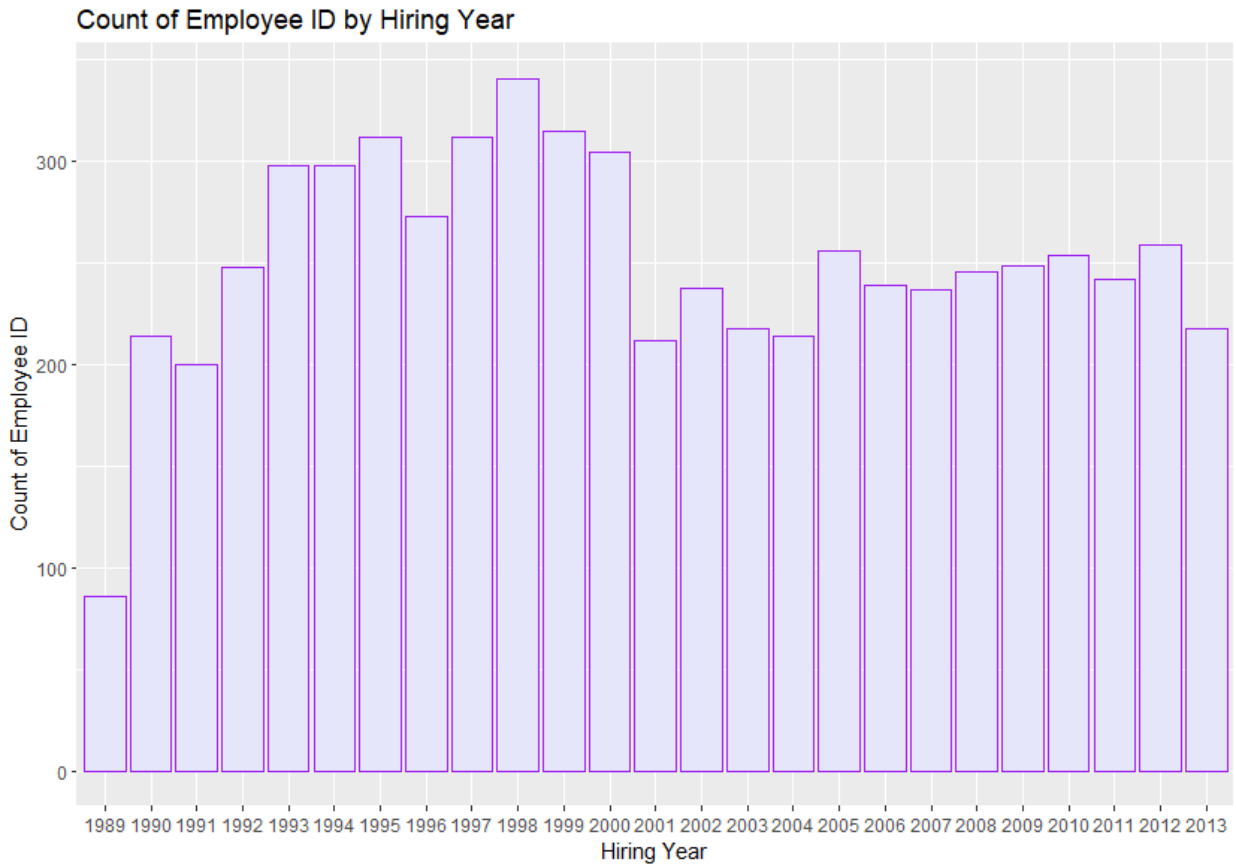
3.9 Bar Graph: Count of Employee ID by Hiring Year

```
# Convert Hired_Date to Date format
data$Hired_Date <- as.Date(data$Hired_Date, format = "%m/%d/%Y")

# Extract year from Hired_Date
data$Hired_Year <- format(data$Hired_Date, format = "%Y")

# Calculate the count of Employee IDs by Hiring Year
count_data <- as.data.frame(table(data$Hired_Year))

# Create a bar plot for Employee_ID count by Hiring_Year
ggplot(data = count_data, aes(x = Var1, y = Freq)) +
  geom_bar(stat = "identity", fill = "#E6E6FA", color = "purple") +
  xlab("Hiring Year") +
  ylab("Count of Employee ID") +
  ggtitle("Count of Employee ID by Hiring Year")
```



Code Explanation:

I used the same code from before for Hired_Year and then used this to `count_data <- as.data.frame(table(data$Hired_Year))`: calculate the count of Employee IDs for each unique hiring year and store the results in the "count_data" dataframe using the `table()` function. `ggplot(data = count_data, aes(x = Var1, y = Freq)) + ...`: is used to create a bar plot. It specifies the data to be used (count_data) and maps the "Hired_Year" to the x-axis (Var1) and the count of Employee IDs to the y-axis (Freq).

Analysis Aim:

To visualize the count of Employee IDs by hiring year by creating a bar plot so that we can understand the distribution of employee hiring over different years and identify any patterns or trends.

Justification:

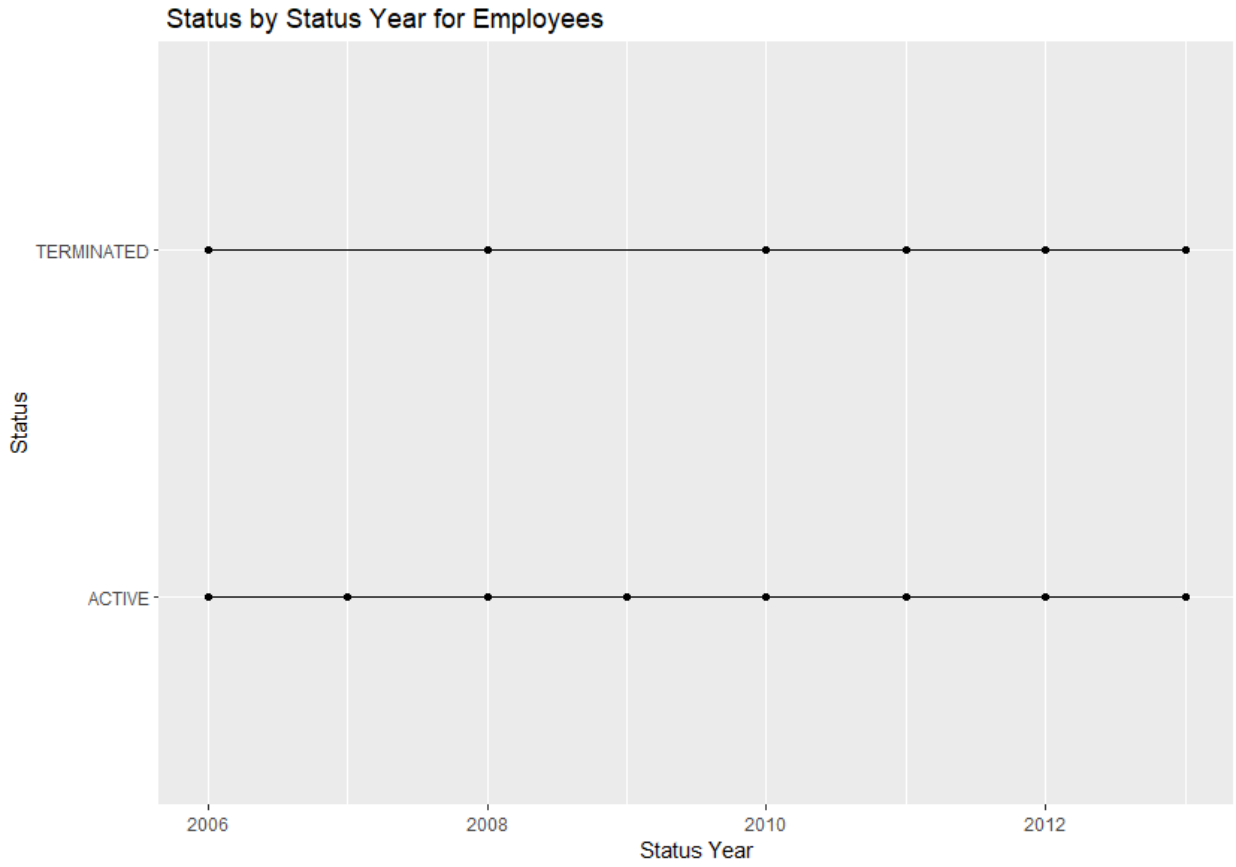
By examining the count of Employee IDs by hiring year, we can gain insights into the hiring trends of the organization. This analysis can help identify periods of high or low hiring activity and understand how the workforce has evolved over time.

Conclusion:

The bar plot provides a visual representation of the count of Employee IDs by hiring year. It shows the distribution of hiring activity over different years. From the plot, we can observe the hiring patterns and identify any significant changes or trends in the organization's hiring practices. This information can be valuable for understanding the growth or contraction of the workforce, identifying recruitment strategies, and evaluating the impact of hiring policies over time.

3.10 Line Plot: Status by Status Year for Employees

```
ggplot(data, aes(x=Status_Year, y=Status)) +
  geom_line()+
  xlab("Status Year")+
  ggtitle("Status by Status Year for Employees")+
  geom_point()
```



Code Explanation:

I created a line plot of the "Status" variable against the "Status_Year" variable from the "data" dataset which also adds data points to the plot. The x-axis is labeled as "Status Year," and the plot title is set as "Status by Status Year for Employees."

Analysis Aim:

To visualize the trend of employee status over different years. By plotting the "Status" variable against the "Status_Year" variable, we can observe how the status of employees has changed over time.

Justification:

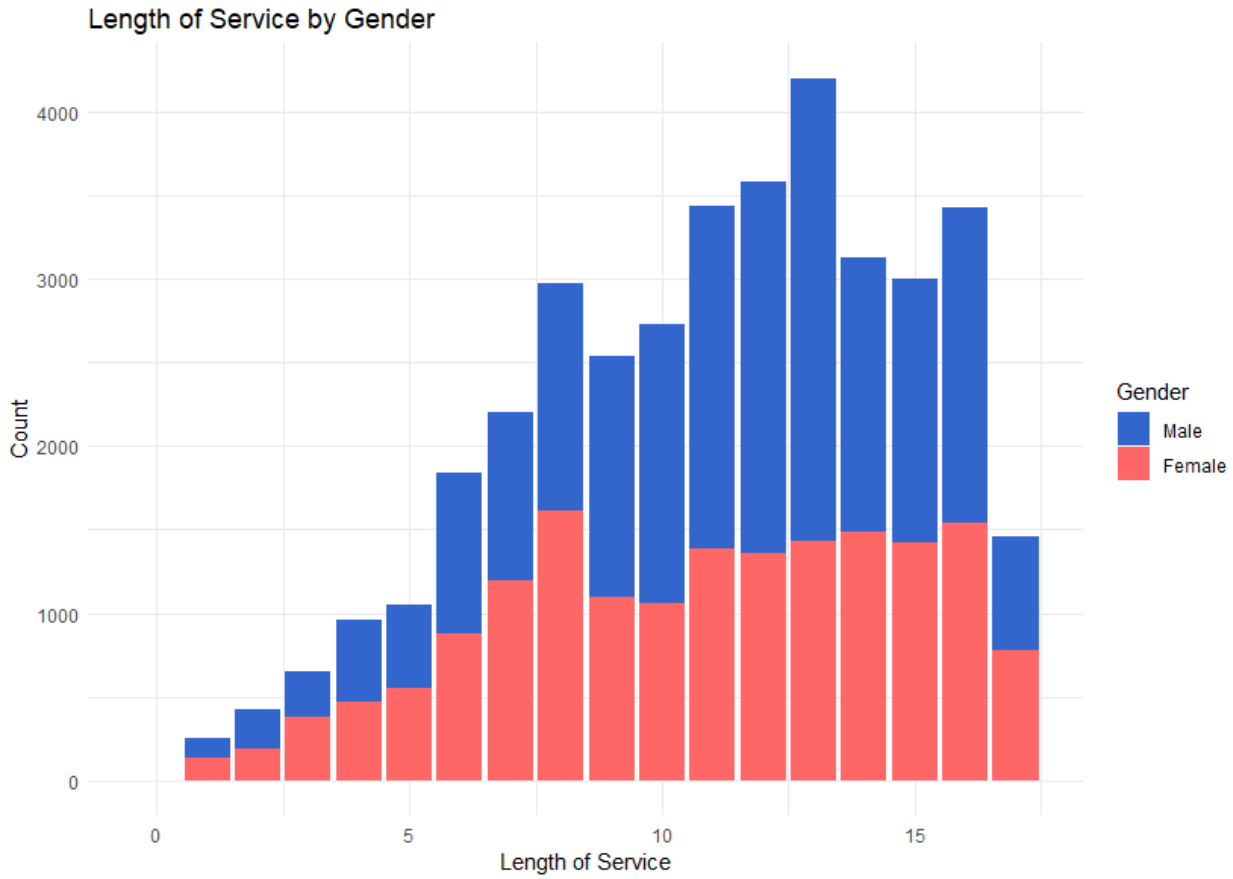
Creating a line plot with data points allows us to visualize the relationship between the "Status" and "Status_Year" variables and shows a visual easy to read. The line plot shows the overall trend, and the data points highlight the individual data values. This approach gives a clear visual of the data.

Conclusion:

The line plot shows the general trend of employee status over time. We can identify any patterns, fluctuations, or significant changes in the status of employees. As you can see the state stays consistent throughout the years and is a straightforward line.

3.11 Stacked Bar Graph: Length of Service by Gender fill

```
ggplot(data, aes(x = Length_of_Service, y= Length_of_Service, fill = Gender)) +
  geom_bar(stat = "identity", position = "stack") +
  labs(title = "Length of Service by Gender",
       x = "Length of Service",
       y = "Count") +
  scale_fill_manual(values = c("#3366CC", "#FF6666"), labels = c("Male", "Female")) +
  theme_minimal()
```



Code Explanation:

I created a bar plot of the "Length_of_Service" variable from the Employee Attrition dataset. The bars are stacked and filled based on the "Gender" variable.

Analysis Aim:

To visualize and compare the distribution of the length of service among employees based on their gender.

Justification:

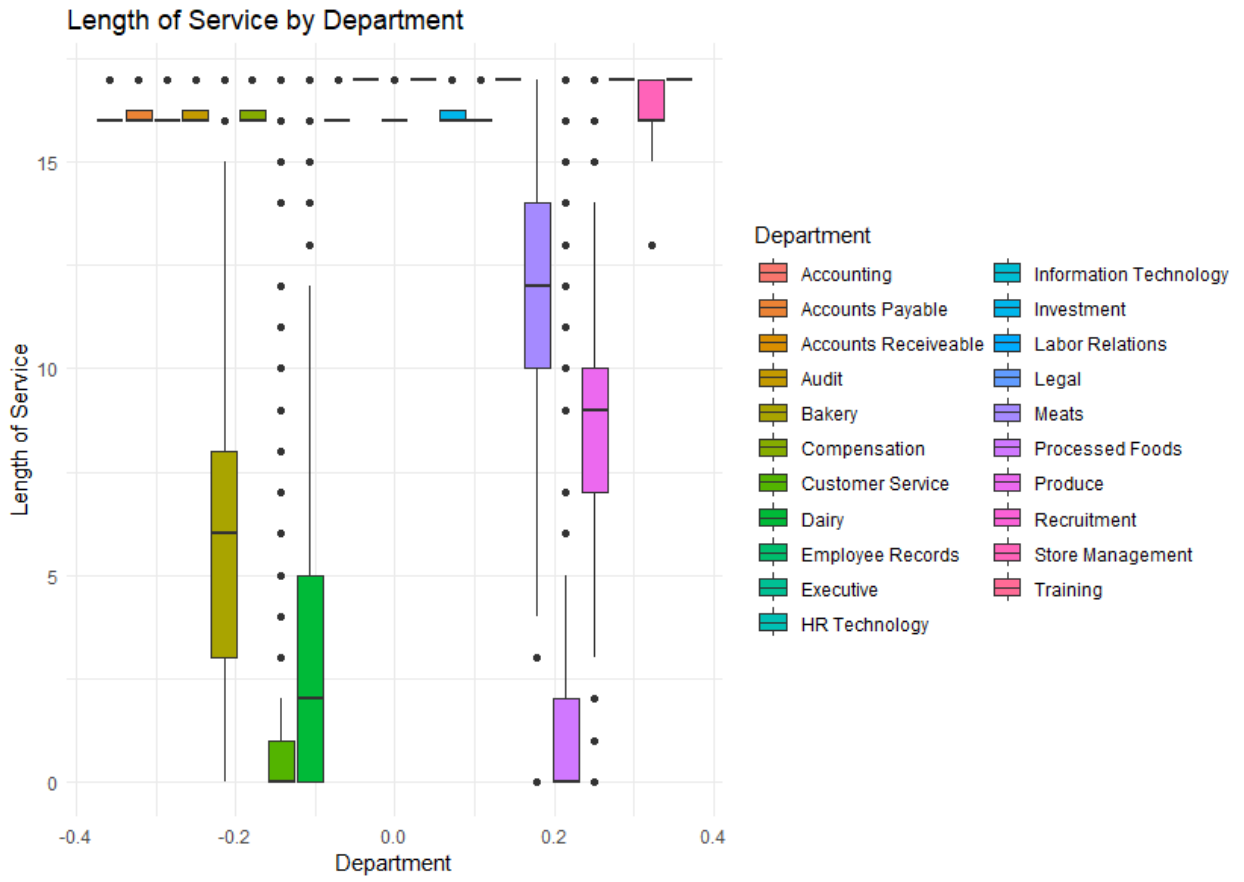
By visualizing the length of service by gender, we can gain insight into any potential differences or patterns that may exist between the genders male and female employees regarding their length of service. This information can provide valuable insights into employee retention, career progression, and potential gender-related factors that may influence tenure within the organization.

Conclusion:

The analysis reveals the distribution of the length of service by gender. The x-axis represents the length of service, and the y-axis represents the number of employees. The bars are stacked and filled with different colors, representing male and female employees.

3.12 Boxplot: Length of Service by Department

```
ggplot(data, aes(y = Length_of_Service, fill = Department)) +
  geom_boxplot() +
  labs(title = "Length of Service by Department", x = "Department", y = "Length of Service") +
  theme_minimal()
```



Code Explanation:

Using ggplot2 I have created a simple boxplot showing the Length of Service in the Y-Axis, and the Departments are represented by color in the boxplots.

Analysis Aim:

To examine the distribution of the length of service across different departments.

Justification:

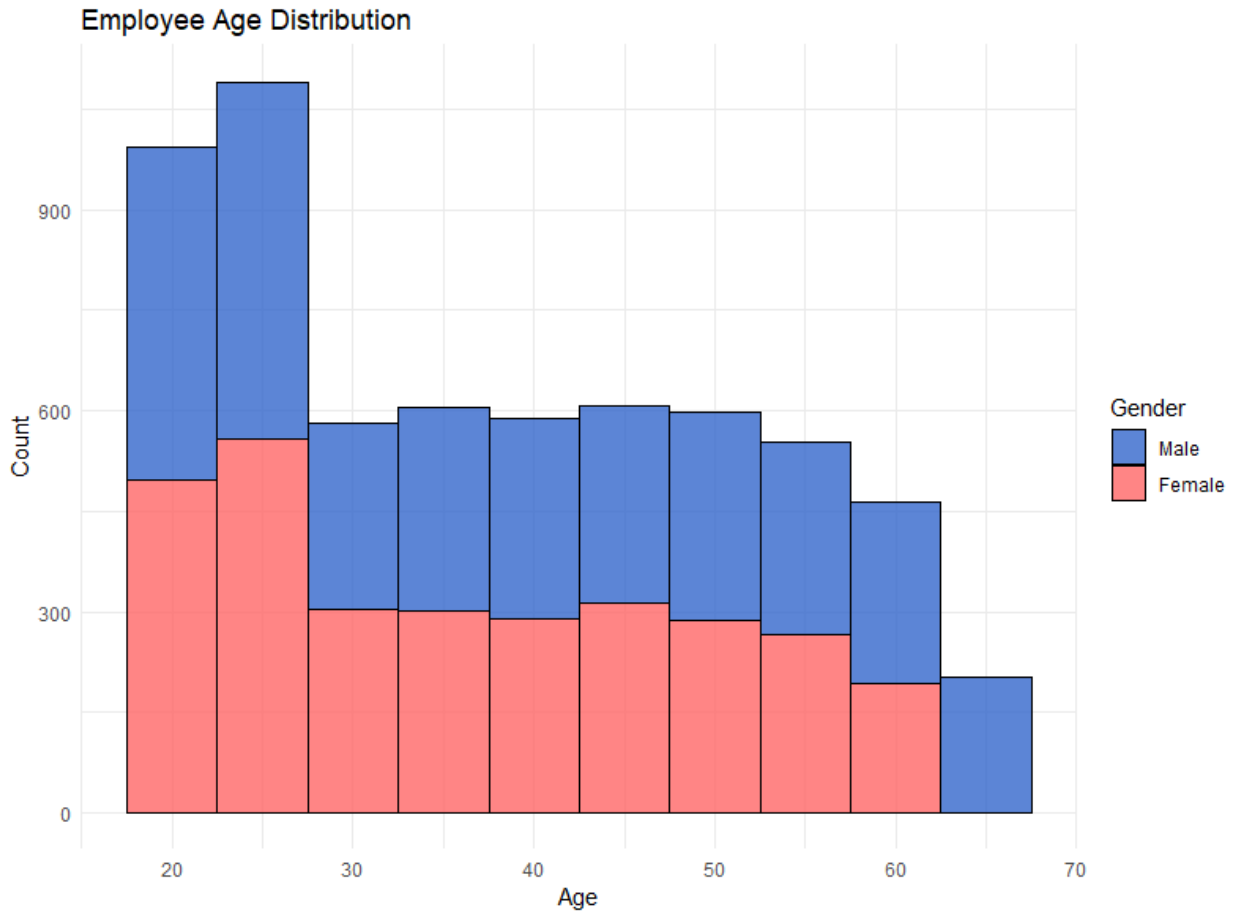
Comparing the length of service across departments can provide insights into employee retention and engagement within specific areas of the organization.

Conclusion:

This box plot is a colorful display of the distribution of the length of service for each department. The x-axis represents the department names, and the y-axis represents the length of service in years. Each boxplot represents the distribution of the length of service within a particular department. This visualization allows for comparisons between departments regarding the distribution of employees.

3.13 Stacked Histogram: Employee Age Distribution

```
ggplot(data, aes(x = Age, fill = Gender)) +
  geom_histogram(binwidth = 5, alpha = 0.8, color = "black") +
  labs(title = "Employee Age Distribution", x = "Age", y = "Count") +
  scale_fill_manual(values = c("#3366CC", "#FF6666"), labels = c("Male", "Female")) +
  theme_minimal()
```



Code Explanation:

Using ggplot2 I created a simple stacked histogram with the Y- axis as counts for the employees and the X-axis as the employees Ages. Pink represents Female and the Blue represents Male in this chart as you can see. This chart is similar code to the histogram I have done previously in this project.

Analysis Aim:

To explore the distribution of employee ages.

Justification:

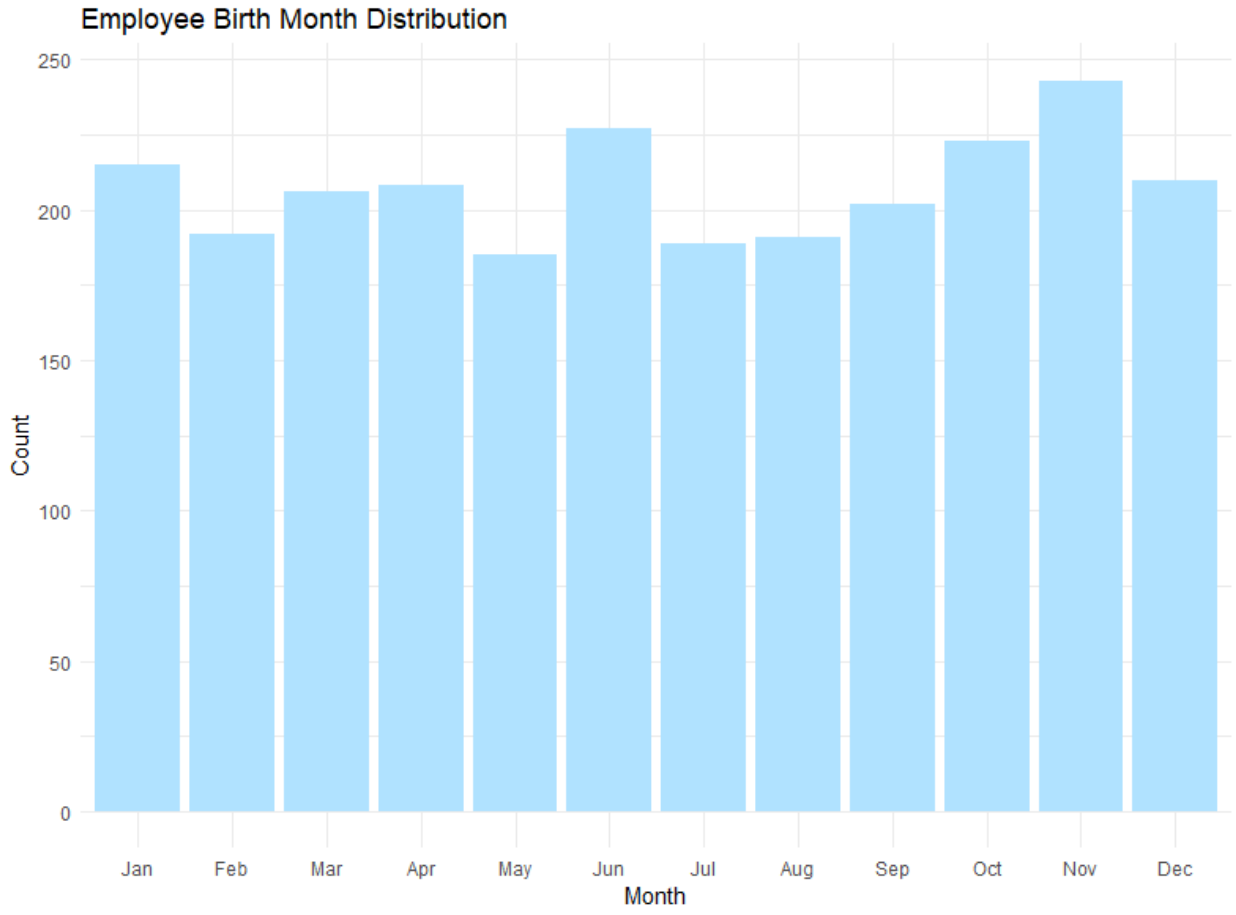
Understanding the age distribution can provide insights into the demographic makeup of the workforce and potential age-related factors that may impact various aspects of the organization.

Conclusion:

The histogram displays the distribution of employee ages. The x-axis represents the age values, and the y-axis represents the count of employees within each age range. The bars are colored differently to indicate the gender of the employees (blue for male, red for female). It allows us to observe the age distribution and identify any gender-related disseminations within different age groups.

3.14 Bar Graph: Employee Birth Month Distribution

```
ggplot(na.omit(data), aes(x = birth_month)) +
  geom_bar(fill = "#B0E2FF") +
  labs(title = "Employee Birth Month Distribution", x = "Month", y = "Count") +
  theme_minimal()
```



Code Explanation:

I created a bar plot of the birth month distribution for all the employees in the data set Employee Attrition. The `na.omit()` function is used to remove rows with NA values in the `birth_month` column before plotting. The `aes()` function is used to specify the x-axis variable (`birth_month`). The `geom_bar()` function is used to create the bar plot, with the fill color set to "#B0E2FF". The `labs()` function is used to set the title, x-axis label, and y-axis label of the plot. Finally, the `theme_minimal()` function is used to set a minimalistic theme for the plot.

Analysis Aim:

To visualize the distribution of employee birth months in the dataset and identify any patterns.

Justification:

By creating a bar plot of the birth month distribution, we can visually examine the frequency of births in each month. Removing the NA values ensures that only valid and complete data points are considered for the analysis.

Conclusion:

The bar chart provides insight into whether certain months have a higher or lower frequency of births among the employees in the dataset.

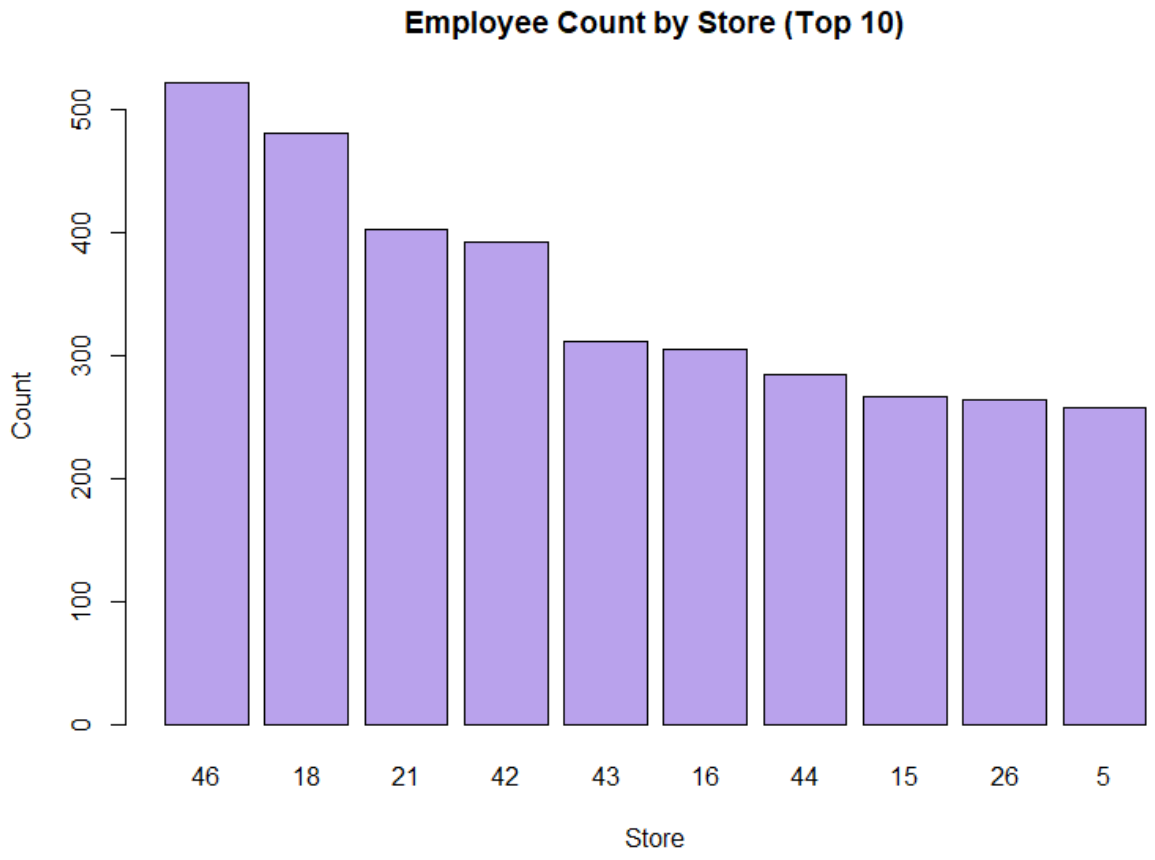
3.15 Bar Graph: Employee count in top 10 stores

```
# Calculate employee count by store
store_counts <- table(data$Store)

# Sort the store counts in descending order
sorted_store_counts <- sort(store_counts, decreasing = TRUE)

# Select the top 10 stores with the highest employee count
top_10_stores <- head(sorted_store_counts, 10)

# Visualize employee count by store (top 10 stores)
barplot(top_10_stores, col = "#B9A2EC", border = "black",
        xlab = "Store", ylab = "Count",
        main = "Employee Count by Store (Top 10)")
```



Code Explanation:

The first step I do is calculating the count of employees for each store using the `table()` function. Next, I sort the store counts in descending order to identify the stores with the highest number of employees. Then, select the top 10 stores with the highest employee count. Finally, I created a bar plot to visually represent the employee count for these top 10 stores.

Analysis Aim:

To understand the distribution of employees across different stores and identify the top 10 stores with the highest number of employees.

Justification:

By examining the employee count by store, it is possible to gain better insight into which stores have the largest workforce. This information is valuable for resource allocation, performance evaluation, and identifying areas where additional support may be required.

Conclusion:

The bar plot showcases the employee count for the top 10 stores with the highest number of employees. By visually representing the data it is easier to identify the stores that have the most employees. This analysis provides a quick overview of the distribution of employees across stores, aiding decision-making, and resource management in HR.

4. Extra/Additional Features

4.1 Pie Chart: Top 5 departments and their employee count

```
# Calculate the count of each department
department_counts <- table(data$Department)

# Sort the department counts in descending order
sorted_counts <- sort(department_counts, decreasing = TRUE)

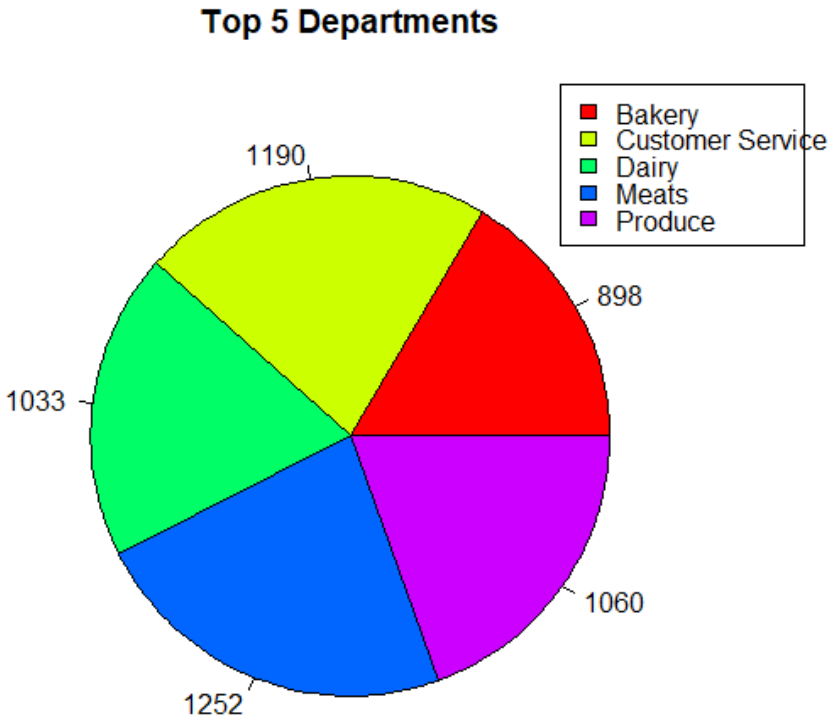
# Select the top 5 departments
top_departments <- names(sorted_counts)[1:5]

# Filter the data for the top 5 departments
top_department_data <- data[data$Department %in% top_departments, ]

# Count the occurrences of the top 5 departments
top_department_counts <- table(top_department_data$Department)

# Create a pie chart for the top 5 departments
pie(top_department_counts, labels = top_department_counts,
    col = rainbow(length(top_department_counts)),
    main = "Top 5 Departments")

# Add a legend
legend("topright", legend = names(top_department_counts),
       fill = rainbow(length(top_department_counts)))
```



Code Explanation:

The code calculates the count of employees in each department using the `table()` function. It sorts the department counts in descending order using `sort()` and stores the result in the `sorted_counts` variable. It selects the top 5 departments based on the sorted counts using the `names()` function and indexing (`[1:5]`). The data is filtered to include only the rows corresponding to the top 5 departments using subsetting (`data$Department %in% top_departments`). The filtered data is then used to calculate the counts of the top 5 departments using `table()`, stored in `top_department_counts`. Finally, a pie chart is created using `pie()` with the counts of the top 5 departments, and a legend is added using `legend()`.(R-Coder, 2020c).

Analysis Aim:

To visualize the distribution of employees among the top 5 departments in the Employee Attrition dataset.

Justification:

By creating a pie chart, we can easily compare the relative sizes of the top 5 departments and gain insight into the employee counts. The use of colors and a legend helps in understanding easier.

Conclusion:

Based on the analysis of the dataset, the added extra feature which is the pie chart shows the distribution of employees among the top 5 departments. The chart allows us to see the proportion of employees in each department visually. This information can be useful for identifying departments with a significant number of employees and gaining a high-level overview of the departmental composition in the organization.

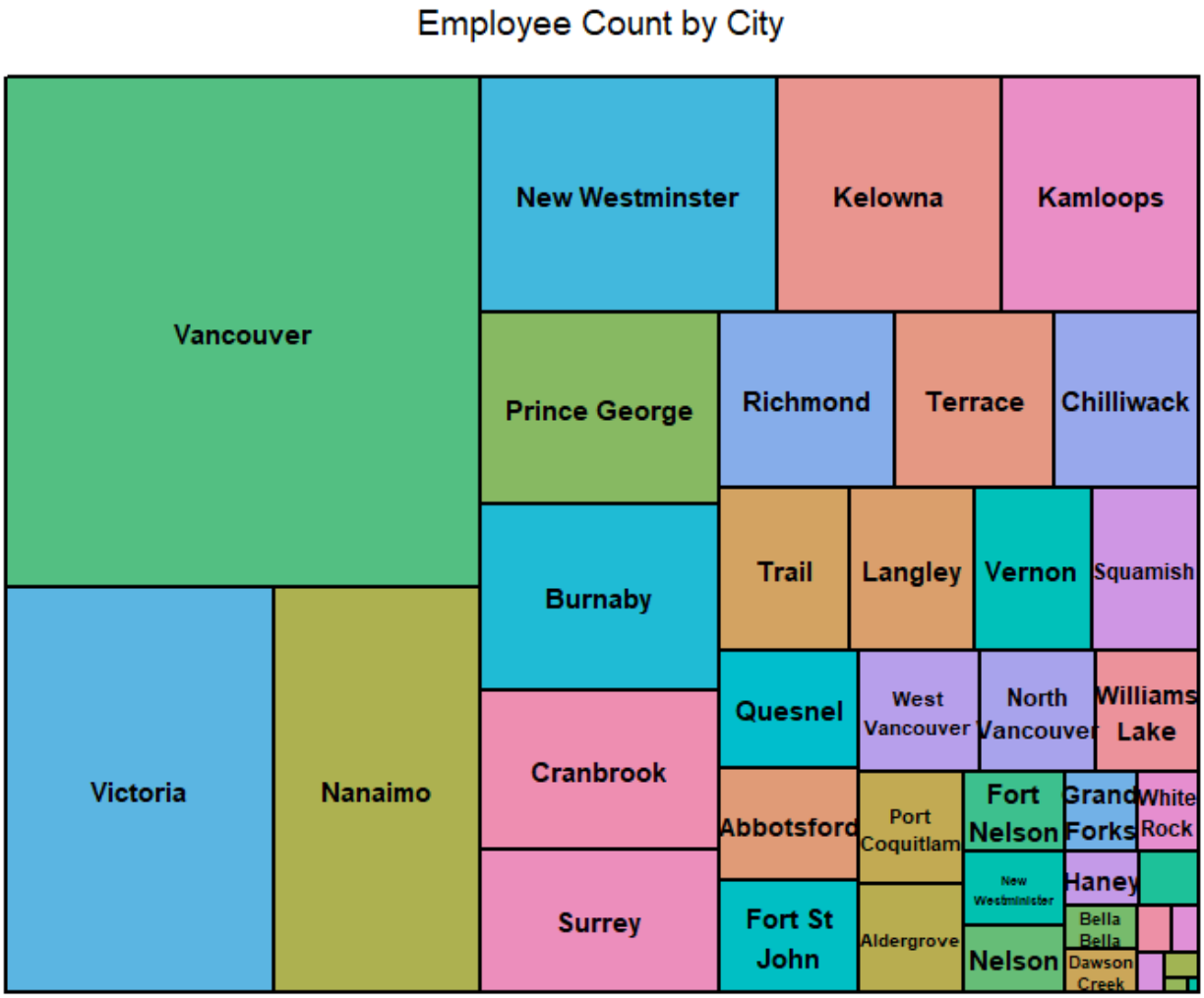
4.2 Treemap: Count of Employees in City's

```
# Calculate the count of employees in each city
city_counts <- table(data$City)

# Convert city_counts to a data frame
city_counts_df <- as.data.frame(city_counts)

# Rename the columns
colnames(city_counts_df) <- c("City", "Count")

# Create the treemap
treemap(city_counts_df, index = "City", vSize = "Count", title = "Employee Count by City")
```



Code Explanation:

First I calculate the count of employees in each city using the `table()` function on the "City" variable in the dataset. Then, convert the resulting table into a data frame using the `as.data.frame()` function. Next, rename the columns of the data frame to "City" and "Count" using the `colnames()` function. Finally, then create a treemap visualization using the `treemap()` function from the "treemap" package. The treemap represents the count of employees in each city, where the size of each rectangle represents the count. (R-Charts, 2021).

Analysis Aim:

To understand the distribution of employees across different cities and visualize it using a treemap aids in identifying cities with a higher concentration of employees and gaining insights into the geographical representation of the employee population from just one look.

Justification:

By analyzing the count of employees in each city, we can identify cities with many employees, potentially indicating important areas with high employee concentration. This information can be useful for various purposes such as resource allocation, understanding employee demographics, and making informed business decisions.

Conclusion:

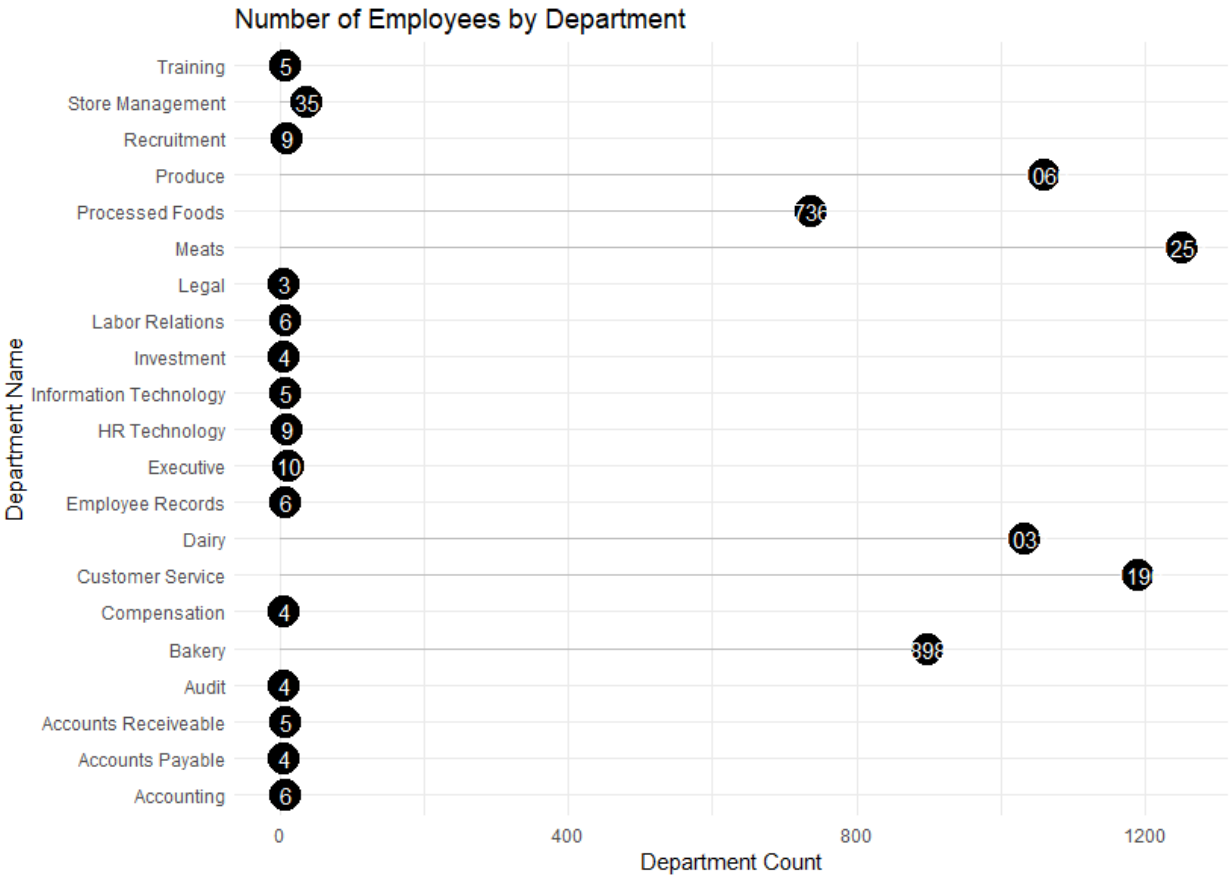
The added extra part is the treemap's visuals of employee counts by city, we can observe the distribution of employees across different cities. The size of each rectangle in the treemap represents the count of employees in that particular city. Cities with larger rectangles indicate a higher number of employees, while smaller rectangles represent cities with a lower count. This analysis provides an overview of the geographical distribution of employees and can serve as a starting point for further investigations or decision-making related to workforce management, recruitment, or resource allocation. Treemaps allow an easy visual guide for analyses like these.

4.3 Lollipop Chart: Number of Employees by Department

```
# Calculate count of employees in each department
department_counts <- table(data$Department)

# Convert department counts to data frame
df_department <- data.frame(Department = names(department_counts),
                            Department_Count = as.numeric(department_counts))

# Lollipop chart
ggplot(df_department, aes(x = Department_Count, y = Department)) +
  geom_segment(aes(x = 0, xend = Department_Count, y = Department, yend = Department),
               color = "gray", size = 0.5) +
  geom_point(size = 7) +
  geom_text(aes(label = Department_Count), color = "white", size = 4, nudge_x = 2) +
  labs(title = "Number of Employees by Department",
       x = "Department Count",
       y = "Department Name") +
  theme_minimal()
```



Code Explanation:

First I calculate the count of employees in each department using the `table()` function. Then, convert the department counts into a data frame (`df_department`) with two columns: Department and Department_Count. The code uses the ggplot2 library to create a lollipop chart to visualize the number of employees in each department. The chart is created using `ggplot()` and the data from `df_department`. `geom_segment()` is used to draw the lines, `geom_point()` to add the points, and `geom_text()` to display the count labels. `labs()` function is used to set the title, x-axis label, and y-axis label of the plot.`theme_minimal()` function is used to apply a minimalistic theme to the plot.

Analysis Aim:

To visualize and understand the distribution of employees across different departments in the organization.

Justification:

Using a lollipop chart is an effective way to display the count of employees in each department as it allows for easy comparison between departments and provides a clear visual representation of the distribution.

Conclusion:

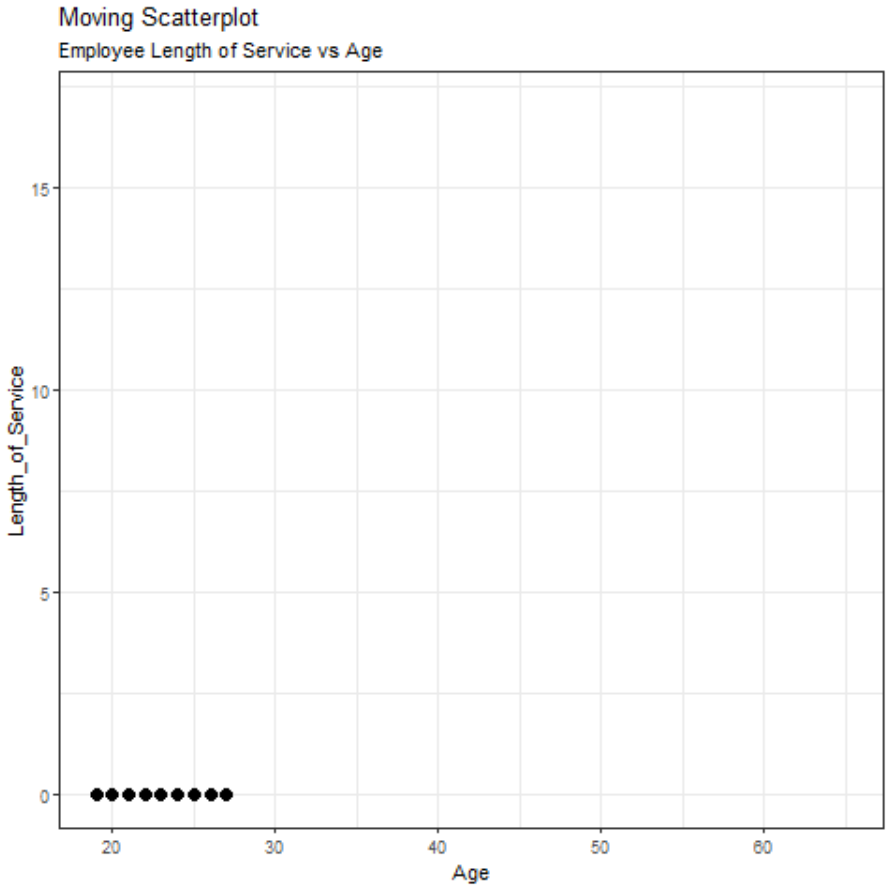
From the lollipop chart, we can observe the number of employees in each department. The length of the lines represents the number of employees, with larger lines indicating a higher number of employees. The chart helps us identify departments with a larger workforce and those with a smaller workforce. It provides a quick overview of the employee distribution across different departments in the organization.

4.4 Moving Scatterplot: Employee Length of Service vs Age

```
# Create animated scatterplot
theme_set(theme_bw())  # Pre-set the bw theme.
g <- ggplot(data, aes(x = Age, y = Length_of_Service)) +
  labs(subtitle = "Employee Length of Service vs Age",
       title = "Moving Scatterplot")

# Animate the scatterplot
p <- g + geom_point(size = 3) +
  transition_time(Length_of_Service) +
  ease_aes("linear")

# Save the animation as a GIF
anim <- animate(p, nframes = 50, fps = 10)
anim_save("Scatterplot.gif", anim)
```



Code Explanation:

The `ggplot()` function initializes the scatterplot, specifying the x-axis as "Age" and the y-axis as "Length_of_Service" from the "data" dataset. The `labs()` function then, sets the subtitle and title of the scatterplot and the `geom_point()` function adds the points to the scatterplot.`transition_time()` function animates the scatterplot based on the "Length_of_Service" variable. `ease_aes()` function defines the animation transition type. `animate()` function generates the animation with a specified number of frames and frames per second. Lastly, the `anim_save()` function saves the animation as a GIF file. (Alboukadel, 2019) (Cooks, n.d.).

Analysis Aim:

To visually explore the relationship between the length of service and age of employees in the dataset.

Justification:

Creating an animated scatterplot allows us to observe how the relationship between length of service and age changes over time in a more visually appealing way. This visualization can help identify any patterns or trends in the data and provide insights into the dynamics in the dataset Employee Attrition.

Conclusion:

Based on the animated scatterplot, we can observe the movement of individual data points representing employees over time. The added advantage of this is the appealing moving scatterplot that is a gif, and this visualization allows us to see the distribution and potential areas of the employees based on their age and length of service.

.

5. Conclusion

In conclusion, I have performed several analyses using the provided dataset, "Employee Attrition" to support the decision-making process of the HRM team. These analyses involved applying different data analytics techniques, such as preparing the data, exploring its patterns, manipulating variables, and creating visual representations. Throughout my analysis, I delved into the relationships between various factors to gain insights into how people make choices when it comes to regarding employees in a large business.

6. References

Alboukadel. (2019, January 24). *gganimate: How to Create Plots with Beautiful Animation in R*. Datanovia. https://www.datanovia.com/en/blog/gganimate-how-to-create-plots-with-beautiful-animation-in-r/

Bates, C. (2020, July 24). *Tidyverse Basics: Load and Clean Data with R tidyverse Tools*. Dataquest. https://www.dataquest.io/blog/load-clean-data-r-tidyverse/

Chapter 11: Boxplots and Bar Graphs. (n.d.). Ademos.people.uic.edu. Retrieved January 7, 2023, from https://ademos.people.uic.edu/Chapter11.html

Cooks, A. (n.d.). *Building an animation step-by-step with gganimate · Alex Cookson*. Www.alexcookson.com. Retrieved May 28, 2023, from https://www.alexcookson.com/post/2020-10-18-building-an-animation-step-by-step-with-gganimate/

Kumar, V. (2021, October 23). *Tutorial for Line Plot in R using ggplot2 with Examples*. MLK - Machine Learning Knowledge. https://machinelearningknowledge.ai/tutorial-for-line-plot-in-r-using-ggplot2-with-examples/

Nantasenamat, C. (2021, June 20). *How to Create Animated Plots in R*. Medium. https://towardsdatascience.com/how-to-create-animated-plots-in-r-adf53a775961

Prabhakaran, S. (n.d.). *Top 50 ggplot2 Visualizations - The Master List (With Full R Code)*. R-Statistics.co. Retrieved May 26, 2023, from http://r-statistics.co/Top50-Ggplot2-Visualizations-MasterList-R-Code.html#Animated%20Bubble%20Plot

R-Charts. (n.d.). *Colors in R*. R CHARTS | a Collection of Charts and Graphs Made with the R Programming Language. Retrieved May 28, 2023, from https://r-charts.com/colors/

R-Charts. (2021, April 13). *Treemaps in ggplot2 with treemapify*. R CHARTS | a Collection of Charts and Graphs Made with the R Programming Language. https://r-charts.com/part-whole/treemapify/

R-Coder. (2020a, April 22). *SCATTER PLOT in R programming [WITH EXAMPLES]*. R CODER. https://r-coder.com/scatter-plot-r/#Scatter_plot_in_ggplot2

R-Coder. (2020b, May 27). *DOT PLOT in R with the dotchart function [with EXAMPLES]*. R CODER. https://r-coder.com/dot-plot-r/

R-Coder. (2020c, July 8). *PIE CHART in R with pie() function ▷ [WITH SEVERAL EXAMPLES]*. R CODER. https://r-coder.com/pie-chart-r/

Schork, J. (n.d.-a). *Barplot in R (8 Examples) | How to Create Barchart & Bargraph in RStudio*. Statistics Globe. https://statisticsglobe.com/barplot-in-r

Schork, J. (n.d.-b). *Create ggplot2 Histogram in R (7 Examples) | geom_histogram Function*. Statistics Globe. Retrieved May 28, 2023, from https://statisticsglobe.com/ggplot2-histogram-in-r-geom_histogram-function

Schork, J. (2022, May 22). *Data Cleaning in R (Tutorial & 9 Examples) | Preparation Techniques*. Statistics Globe. https://statisticsglobe.com/data-cleaning-r

STHDA. (n.d.-a). *ggplot2 violin plot : Quick start guide - R software and data visualization - Easy Guides - Wiki - STHDA.* Www.sthda.com. http://www.sthda.com/english/wiki/ggplot2-violin-plot-quick-start-guide-r-software-and-data-visualization

STHDA. (n.d.-b). *Strip charts: 1-D scatter plots - R Base Graphs - Easy Guides - Wiki - STHDA.* Www.sthda.com. Retrieved May 28, 2023, from http://www.sthda.com/english/wiki/strip-charts-1-d-scatter-plots-r-base-graphs

Zach. (2019, April 13). *How to Create a Lollipop Chart in R*. Statology. https://www.statology.org/lollipop-chart-r/

Zach. (2020, October 27). *How to Import CSV Files into R (Step-by-Step).* Statology. https://www.statology.org/import-csv-into-r/